

Max ▾



Perfekt — ich habe alle aktuellen APIs gecheckt. Jetzt liefere ich **Option C**: beides auf einmal — vollständig, produktionsreif, copy-paste-bereit.



{ } text

NEU in diesem Schritt

- Tray-Icon → App lebt im System-Tray, nicht nur offen
- Autostart → `app.setLoginItemSettings` (offiziell API)
- Notifications → native Windows-Benachrichtigungen
- Hermes Panel → Quick Connect / Model-Status / API-Test
- Settings-Page → Persistente UI für alle Optionen
- AI-Studio-Prompt → Präziser Übergabe-Prompt für jede KI

1 main.js

{ } JavaScript

```
const {
  app, BrowserWindow, ipcMain, dialog,
  shell, Tray, Menu, nativeImage, Notification
} = require('electron')
const path = require('path')
const Store = require('electron-store')

const store = new Store()

// — Globale Referenzen (nie GC-en lassen!) —————
let mainWindow = null
let tray = null
let llamaProcess = null
```

```

let logBuffer    = []
let serverRunning = false

// =====
// TRAY-ICON (inline Base64 – kein externes Asset nötig)
// 16x16 PNG in Base64: Hermes-Symbol (lila Gehirn-Icon)
// =====
function getTrayIcon(active = false) {
  // Minimales 16x16 Icon als NativeImage – ersetzbar durch echte .ico-Datei
  const iconPath = path.join(__dirname, 'assets',
    active ? 'tray-active.png' : 'tray-idle.png')

  try {
    return nativeImage.createFromPath(iconPath)
  } catch {
    // Fallback: 1px transparentes PNG
    return nativeImage.createFromDataURL(
      ''
    )
  }
}

// =====
// TRAY ERSTELLEN
// =====
function createTray() {
  tray = new Tray(getTrayIcon(false))
  tray.setToolTip('Hermes Control Center')
  updateTrayMenu()

  // Linksklick → Fenster togglen
  tray.on('click', () => {
    if (mainWindow) {
      mainWindow.isVisible() ? mainWindow.hide() : mainWindow.show()
    }
  })
}

function updateTrayMenu() {
  if (!tray) return

  const menu = Menu.buildFromTemplate([
    {
      label: '🧠 Hermes Control Center',
      enabled: false,
    },
    { type: 'separator' },
    {
      label: serverRunning ? '🟢 Server läuft' : '🔴 Server gestoppt',
      enabled: false
    },
    {
      label: serverRunning ? '■ Server stoppen' : '▶ Server starten',
      click: () => {
        if (serverRunning) {

```

```

        if (llamaProcess) llamaProcess.kill('SIGTERM')
      } else {
        // Fenster öffnen und Nutzer starten lassen
        mainWindow?.show()
        mainWindow?.webContents.send('tray-start-request')
      }
    },
    { type: 'separator' },
    {
      label: '🌐 API Öffnen',
      enabled: serverRunning,
      click: () => {
        const port = store.get('port', 8080)
        shell.openExternal(`http://127.0.0.1:${port}`)
      }
    },
    {
      label: '🪟 Fenster öffnen',
      click: () => {
        mainWindow?.show()
        mainWindow?.focus()
      }
    },
    { type: 'separator' },
    {
      label: '⚙ Autostart',
      type: 'checkbox',
      checked: app.getLoginItemSettings().openAtLogin,
      click: (menuItem) => {
        toggleAutostart(menuItem.checked)
      }
    },
    { type: 'separator' },
    {
      label: '✕ Beenden',
      click: () => {
        if (llamaProcess) llamaProcess.kill('SIGTERM')
        app.quit()
      }
    }
  ])

  tray.setContextMenu(menu)
  tray.setImage(getTrayIcon(serverRunning))
}

// =====
// AUTOSTART – offizielles Electron-API
// Quelle: app.setLoginItemSettings (cross-platform, kein VBScript)
// =====
function toggleAutostart(enable) {
  if (process.platform === 'win32') {
    // Windows: Pfad zum tatsächlichen Executable
    const exePath = process.execPath

    app.setLoginItemSettings({

```

```

    openAtLogin: enable,
    path: exePath,
    args: ['--autostart'] // Flag damit App weiß: via Login gestartet
  })
} else {
  // macOS / Linux
  app.setLoginItemSettings({ openAtLogin: enable })
}

store.set('autostart', enable)

// Tray-Menü aktualisieren
updateTrayMenu()

showNotification(
  enable ? 'Autostart aktiviert' : 'Autostart deaktiviert',
  enable
    ? 'Hermes startet automatisch beim Windows-Start.'
    : 'Hermes startet nicht mehr automatisch.'
)
}

// =====
// NATIVE NOTIFICATIONS
// =====
function showNotification(title, body, urgent = false) {
  if (!Notification.isSupported()) return

  const n = new Notification({
    title,
    body,
    // icon: path.join(__dirname, 'assets', 'icon.png'),
    urgency: urgent ? 'critical' : 'normal',
    timeoutType: 'default',
    silent: !urgent
  })

  n.on('click', () => {
    mainWindow?.show()
    mainWindow?.focus()
  })

  n.show()
}

// =====
// FENSTER
// =====
function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1200,
    height: 800,
    minWidth: 900,
    minHeight: 600,
    backgroundColor: '#0f0f0f',
    titleBarStyle: 'hidden',
    trafficLightPosition: { x: 16, y: 16 },
  })
}

```

```

    show: false, // Erst zeigen wenn ready (verhindert Flash)
  webPreferences: {
    preload: path.join(__dirname, 'preload.js'),
    contextIsolation: true,
    nodeIntegration: false,
    sandbox: false
  }
})

mainWindow.loadFile('renderer/index.html')

// Beim Schließen: in Tray minimieren statt beenden
mainWindow.on('close', (event) => {
  if (!app.isQuitting) {
    event.preventDefault()
    mainWindow.hide()
    showNotification(
      'Hermes läuft im Hintergrund',
      'Klicke auf das Tray-Icon um die App zu öffnen.'
    )
  }
})

// Fenster erst zeigen wenn fertig geladen (kein weißer Flash)
mainWindow.once('ready-to-show', () => {
  // Beim Autostart: versteckt starten
  const wasAutostarted = process.argv.includes('--autostart')
  if (!wasAutostarted) {
    mainWindow.show()
  }
})
}

// =====
// APP LIFECYCLE
// =====

app.whenReady().then(() => {
  createWindow()
  createTray()
})

app.on('before-quit', () => {
  app.isQuitting = true
  if (llamaProcess) llamaProcess.kill('SIGTERM')
})

// Wichtig: window-all-closed NICHT app.quit() callen
// damit Tray-only-Modus funktioniert
app.on('window-all-closed', () => {
  // Absichtlich leer – App lebt im Tray weiter
})

app.on('activate', () => {
  // macOS: Dock-Klick
  mainWindow?.show()
})

```

```

// =====
// IPC HANDLER – Dateiauswahl
// =====

ipcMain.handle('select-llama-exe', async () => {
  const result = await dialog.showOpenDialog(mainWindow, {
    title: 'llama-server.exe auswählen',
    buttonLabel: 'Auswählen',
    defaultPath: store.get('llamaExePath', 'C:\\\\'),
    filters: [
      { name: 'Executables', extensions: ['.exe'] },
      { name: 'Alle Dateien', extensions: ['*'] }
    ],
    properties: ['openFile']
  })
  if (!result.canceled && result.filePaths[0]) {
    store.set('llamaExePath', result.filePaths[0])
    return { success: true, path: result.filePaths[0] }
  }
  return { success: false }
})

ipcMain.handle('select-gguf-model', async () => {
  const result = await dialog.showOpenDialog(mainWindow, {
    title: 'GGUF-Modell auswählen',
    buttonLabel: 'Modell laden',
    defaultPath: store.get('ggufModelPath', 'C:\\\\'),
    filters: [
      { name: 'GGUF Modelle', extensions: ['.gguf'] },
      { name: 'Alle Dateien', extensions: ['*'] }
    ],
    properties: ['openFile']
  })
  if (!result.canceled && result.filePaths[0]) {
    store.set('ggufModelPath', result.filePaths[0])
    return { success: true, path: result.filePaths[0] }
  }
  return { success: false }
})

// =====
// IPC HANDLER – Einstellungen
// =====

ipcMain.handle('get-saved-paths', () => ({
  llamaExePath: store.get('llamaExePath', ''),
  ggufModelPath: store.get('ggufModelPath', ''),
  port: store.get('port', 8080),
  ngl: store.get('ngl', 35),
  ctxSize: store.get('ctxSize', 4096),
  threads: store.get('threads', 8),
  autostart: store.get('autostart', false),
  minimizeToTray: store.get('minimizeToTray', true),
  theme: store.get('theme', 'dark'),
}))

ipcMain.handle('save-settings', (_, settings) => {
  Object.entries(settings).forEach(([k, v]) => store.set(k, v))
})

```

```

// Autostart direkt anwenden wenn geändert
if (settings.autostart !== undefined) {
  toggleAutostart(settings.autostart)
}
return { success: true }
})

ipcMain.handle('get-autostart-status', () =>
  app.getLoginItemSettings().openAtLogin
)

ipcMain.handle('toggle-autostart', (_, enable) => {
  toggleAutostart(enable)
  return { success: true, enabled: enable }
})

// =====
// IPC HANDLER – Server-Steuerung
// =====
ipcMain.handle('start-llama', async (_, config) => {
  if (llamaProcess) return { success: false, error: 'Server läuft bereits!' }

  const { exePath, modelPath, port, ngl, ctxSize, threads } = config

  if (!exePath || !modelPath)
    return { success: false, error: 'Exe und Modell müssen ausgewählt sein.' }

  const args = [
    '-m',      modelPath,
    '--port',  String(port    || 8080),
    '-ngl',    String(ngl     || 35),
    '-c',      String(ctxSize || 4096),
    '-t',      String(threads || 8),
    '--host',  '127.0.0.1'
  ]

  const { spawn } = require('child_process')

  try {
    llamaProcess = spawn(exePath, args, {
      env: { ...process.env },
      windowsHide: true
    })

    const pushLog = (level, msg) => {
      const entry = { ts: Date.now(), level, msg: msg.trim() }
      logBuffer.push(entry)
      if (logBuffer.length > 5000) logBuffer.shift()
      mainWindow?.webContents.send('log-line', { level, msg: msg.trim() })
    }

    llamaProcess.stdout.on('data', d =>
      d.toString().split('\n').filter(l=>l.trim()).forEach(l => pushLog('info', l))
    )

    llamaProcess.stderr.on('data', d =>
      d.toString().split('\n').filter(l=>l.trim()).forEach(l => {

```

```

const level = l.toLowerCase().includes('error') ? 'error' : 'warn'
pushLog(level, l)

// Erkennung: Server ist bereit
if (l.includes('HTTP server listening') || l.includes('llama server listening')) {
  serverRunning = true
  updateTrayMenu()
  showNotification(
    '🚀 Hermes Server gestartet',
    `Modell geladen auf Port ${port || 8080}`
  )
  mainWindow?.webContents.send('server-ready', { port: port || 8080 })
}
})
)

llamaProcess.on('close', (code) => {
  serverRunning = false
  updateTrayMenu()
  pushLog('warn', `llama-server beendet (Exit-Code: ${code})`)
  mainWindow?.webContents.send('server-stopped', { code })

  if (code !== 0 && code !== null) {
    showNotification('⚠️ Server abgestürzt', `Exit-Code: ${code}`, true)
  }

  llamaProcess = null
})

llamaProcess.on('error', (err) => {
  mainWindow?.webContents.send('log-line', {
    level: 'error', msg: `Spawn-Fehler: ${err.message}`
  })
  llamaProcess = null
  serverRunning = false
  updateTrayMenu()
})

return { success: true }

} catch (err) {
  return { success: false, error: err.message }
}
})

ipcMain.handle('stop-llama', () => {
  if (llamaProcess) {
    llamaProcess.kill('SIGTERM')
    return { success: true }
  }
  return { success: false, error: 'Kein Prozess aktiv.' }
})

ipcMain.handle('get-log-history', () => logBuffer)
ipcMain.handle('get-server-status', () => ({ running: serverRunning }))

// =====

```



```
// IPC HANDLER – System / Explorer / Notifications
// =====

ipcMain.handle('open-in-explorer', (_, filePath) => {
  shell.showItemInFolder(filePath)
  return { success: true }
})

ipcMain.handle('open-external-url', (_, url) => {
  shell.openExternal(url)
  return { success: true }
})

ipcMain.handle('show-notification', (_, { title, body, urgent }) => {
  showNotification(title, body, urgent)
  return { success: true }
})

// API Health-Check (für Hermes Quick Connect Panel)
ipcMain.handle('api-health-check', async (_, port) => {
  const http = require('http')
  return new Promise((resolve) => {
    const req = http.get(
      `http://127.0.0.1:${port || 8080}/health`,
      { timeout: 3000 },
      (res) => {
        let data = ''
        res.on('data', d => data += d)
        res.on('end', () => {
          try {
            resolve({ success: true, status: JSON.parse(data) })
          } catch {
            resolve({ success: true, status: { status: 'ok' } })
          }
        })
      }
    )
    req.on('error', (err) => resolve({ success: false, error: err.message }))
    req.on('timeout', () => {
      req.destroy()
      resolve({ success: false, error: 'Timeout' })
    })
  })
})
})
```

2 preload.js

{ } JavaScript

```
const { contextBridge, ipcRenderer } = require('electron')

contextBridge.exposeInMainWorld('electronAPI', {
```

```

// — Dateiauswahl —————
selectLlamaExe:      () => ipcRenderer.invoke('select-llama-exe'),
selectGgufModel:     () => ipcRenderer.invoke('select-gguf-model'),

// — Einstellungen —————
getSavedPaths:       () => ipcRenderer.invoke('get-saved-paths'),
saveSettings:        (s) => ipcRenderer.invoke('save-settings', s),

// — Autostart —————
getAutostartStatus:  () => ipcRenderer.invoke('get-autostart-status'),
toggleAutostart:     (e) => ipcRenderer.invoke('toggle-autostart', e),

// — Server —————
startLlama:          (c) => ipcRenderer.invoke('start-llama', c),
stopLlama:           () => ipcRenderer.invoke('stop-llama'),
getServerStatus:     () => ipcRenderer.invoke('get-server-status'),

// — API Health-Check —————
apiHealthCheck:      (p) => ipcRenderer.invoke('api-health-check', p),

// — Logs —————
getLogHistory:       () => ipcRenderer.invoke('get-log-history'),

// — System —————
openInExplorer:      (p) => ipcRenderer.invoke('open-in-explorer', p),
openExternalUrl:     (u) => ipcRenderer.invoke('open-external-url', u),
showNotification:    (o) => ipcRenderer.invoke('show-notification', o),

// — Event-Listener (Renderer ← Main) —————
onLogLine:           (cb) => ipcRenderer.on('log-line',      (_, d) => cb(d)),
onServerReady:       (cb) => ipcRenderer.on('server-ready',  (_, d) => cb(d)),
onServerStop:        (cb) => ipcRenderer.on('server-stopped', (_, d) => cb(d)),
onTrayStartRequest:  (cb) => ipcRenderer.on('tray-start-request', () => cb()),

// — Cleanup —————
removeListener:       (ch, cb) => ipcRenderer.removeListener(ch, cb),
removeAllListeners:   (ch) => ipcRenderer.removeAllListeners(ch)
})

```

3 renderer/index.html

{ } HTML

```

<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="Content-Security-Policy"
    content="default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'">
  <title>Hermes Control Center</title>

```

```

<link rel="stylesheet" href="styles.css">
</head>
<body>

<!-- == TITLEBAR == -->
<div class="titlebar">
  <span class="titlebar-logo">🟡</span>
  <span class="titlebar-title">Hermes Control Center</span>
  <div class="titlebar-status">
    <span id="global-dot" class="dot dot-off"></span>
    <span id="global-status">Gestoppt</span>
  </div>
  <div class="titlebar-actions">
    <button class="tb-btn" id="tb-minimize" title="Minimieren">-</button>
    <button class="tb-btn" id="tb-tray" title="In Tray">⏏</button>
  </div>
</div>

<!-- == NAV-SIDEBAR == -->
<div class="app-layout">

  <nav class="sidebar">
    <button class="nav-btn active" data-page="dashboard">
      <span class="nav-icon">🏠</span>
      <span class="nav-label">Dashboard</span>
    </button>
    <button class="nav-btn" data-page="llama">
      <span class="nav-icon">⚙️</span>
      <span class="nav-label">llama-server</span>
    </button>
    <button class="nav-btn" data-page="hermes">
      <span class="nav-icon">🟡</span>
      <span class="nav-label">Hermes Panel</span>
    </button>
    <button class="nav-btn" data-page="logs">
      <span class="nav-icon">📄</span>
      <span class="nav-label">Logs</span>
      <span class="nav-badge" id="error-badge" style="display:none">!</span>
    </button>
    <button class="nav-btn" data-page="settings">
      <span class="nav-icon">⚙️</span>
      <span class="nav-label">Einstellungen</span>
    </button>

    <!-- Sidebar-Bottom -->
    <div class="sidebar-bottom">
      <button class="nav-btn nav-btn-danger" id="nav-quit">
        <span class="nav-icon">✖</span>
        <span class="nav-label">Beenden</span>
      </button>
    </div>
  </nav>

  <!-- == PAGE CONTAINER == -->
  <main class="page-container">

    <!-- == DASHBOARD PAGE == -->

```

```

<section class="page active" id="page-dashboard">
  <div class="page-header">
    <h1>Dashboard</h1>
    <p class="page-subtitle">Übersicht und Schnellzugriff</p>
  </div>

  <!-- Status-Cards -->
  <div class="status-cards">
    <div class="status-card" id="card-server">
      <div class="sc-icon">🖥️</div>
      <div class="sc-body">
        <div class="sc-label">llama-server</div>
        <div class="sc-value" id="dash-server-status">Gestoppt</div>
      </div>
      <div class="sc-indicator dot dot-off" id="dash-server-dot"></div>
    </div>
    <div class="status-card" id="card-model">
      <div class="sc-icon">🧠</div>
      <div class="sc-body">
        <div class="sc-label">Modell</div>
        <div class="sc-value" id="dash-model-name">Nicht geladen</div>
      </div>
    </div>
    <div class="status-card" id="card-api">
      <div class="sc-icon">🌐</div>
      <div class="sc-body">
        <div class="sc-label">API-Endpoint</div>
        <div class="sc-value" id="dash-api-url">—</div>
      </div>
      <button class="sc-action" id="dash-test-api" disabled>Testen</button>
    </div>
    <div class="status-card" id="card-uptime">
      <div class="sc-icon">🕒</div>
      <div class="sc-body">
        <div class="sc-label">Laufzeit</div>
        <div class="sc-value" id="dash-uptime">—</div>
      </div>
    </div>
  </div>

  <!-- Quick Actions -->
  <div class="quick-actions">
    <h2>Schnellaktionen</h2>
    <div class="qa-grid">
      <button class="qa-btn" id="qa-start">
        <span>▶</span>
        <div>
          <strong>Server starten</strong>
          <small>Letztes Profil laden</small>
        </div>
      </button>
      <button class="qa-btn" id="qa-wizard">
        <span>🧙</span>
        <div>
          <strong>Setup-Assistent</strong>
          <small>Geführte Einrichtung</small>
        </div>
      </button>
    </div>
  </div>

```

```

</button>
<button class="qa-btn" id="qa-api">
  <span>🌐</span>
  <div>
    <strong>API öffnen</strong>
    <small>Im Browser öffnen</small>
  </div>
</button>
<button class="qa-btn" id="qa-logs">
  <span>📄</span>
  <div>
    <strong>Logs ansehen</strong>
    <small>Server-Ausgabe</small>
  </div>
</button>
</div>
</div>
</section>

<!-- — LLAMA PAGE — -->
<section class="page" id="page-llama">
  <div class="page-header">
    <h1>⚙️ llama-server Konfiguration</h1>
  </div>

  <div class="two-col">
    <div class="card">
      <h3>📅 Dateipfade</h3>

      <div class="field-group">
        <label>llama-server.exe</label>
        <div class="path-row">
          <input id="exe-path" type="text" readonly
            placeholder="Noch nicht ausgewählt...">
          <button id="btn-exe" class="btn btn-pick">📁</button>
          <button id="btn-exe-open" class="btn btn-icon"
            title="Im Explorer öffnen" disabled>🔍</button>
        </div>
      </div>

      <div class="field-group">
        <label>GGUF-Modell</label>
        <div class="path-row">
          <input id="model-path" type="text" readonly
            placeholder="Noch nicht ausgewählt...">
          <button id="btn-model" class="btn btn-pick">📁</button>
          <button id="btn-model-open" class="btn btn-icon"
            title="Im Explorer öffnen" disabled>🔍</button>
        </div>
      </div>

      <div class="action-row">
        <button id="btn-start" class="btn btn-start" disabled>▶ Starten</button>
        <button id="btn-stop" class="btn btn-stop" disabled>■ Stoppen</button>
      </div>

      <div id="server-info-bar" class="info-bar hidden">

```



```

        <code id="hermes-health">—</code>
    </div>
    <div class="detail-row">
        <span>Modell</span>
        <code id="hermes-model">—</code>
    </div>
</div>
<div class="cs-actions">
    <button id="btn-hermes-connect" class="btn btn-start">
        📶 Verbinden
    </button>
    <button id="btn-hermes-refresh" class="btn btn-pick">
        🔄 Aktualisieren
    </button>
</div>
</div>
</div>

<!-- API-Test ————— -->
<div class="card api-test-card">
    <h3>🔧 API-Schnelltest</h3>
    <div class="api-test">
        <div class="field-group">
            <label>Testnachricht</label>
            <textarea id="api-test-prompt" rows="3"
                placeholder="Hallo! Wer bist du?"></textarea>
        </div>
        <button id="btn-api-test" class="btn btn-start">
            ▶ Test senden
        </button>
        <div class="api-response-box" id="api-response-box">
            <div class="response-placeholder">Antwort erscheint hier...</div>
        </div>
        <div class="api-stats" id="api-stats" style="display:none">
            <span id="stat-time">—ms</span>
            <span id="stat-tokens">— tokens</span>
        </div>
    </div>
</div>

<!-- Modell-Infos ————— -->
<div class="card model-info-card">
    <h3>📄 Modell-Informationen</h3>
    <div id="model-info-content" class="model-info">
        <div class="info-placeholder">
            Verbinde dich zuerst mit dem Server...
        </div>
    </div>
</div>

</div>
</section>

<!-- — LOGS PAGE ————— -->
<section class="page" id="page-logs">
    <div class="page-header">
        <h1>📄 Server-Logs</h1>

```

```

<div class="log-toolbar">
  <input id="log-search" type="text"
    placeholder="🔍 Suchen..." class="log-search-input">
  <label class="toggle-label">
    <input type="checkbox" id="filter-errors"> Nur Fehler
  </label>
  <label class="toggle-label">
    <input type="checkbox" id="filter-scroll" checked=""> Auto-Scroll
  </label>
  <button id="btn-clear-log" class="btn btn-sm">🗑 Leeren</button>
  <button id="btn-export-log" class="btn btn-sm">📄 Export</button>
</div>
</div>

<div id="log-viewer" class="log-viewer-full">
  <div class="log-placeholder">Starte den Server um Logs zu sehen...</div>
</div>

<div class="log-footer">
  <span id="log-count">0 Einträge</span>
  <span id="log-footer-hint"></span>
</div>
</section>

<!-- — SETTINGS PAGE — -->
<section class="page" id="page-settings">
  <div class="page-header">
    <h1>⚙ Einstellungen</h1>
  </div>

  <div class="settings-grid">

    <!-- System — -->
    <div class="card settings-card">
      <h3>🖥 System</h3>
      <div class="setting-row">
        <div class="setting-info">
          <strong>Autostart</strong>
          <small>Mit Windows starten</small>
        </div>
        <label class="toggle-switch">
          <input type="checkbox" id="setting-autostart">
          <span class="slider"></span>
        </label>
      </div>
      <div class="setting-row">
        <div class="setting-info">
          <strong>In Tray minimieren</strong>
          <small>Beim Schließen in Tray</small>
        </div>
        <label class="toggle-switch">
          <input type="checkbox" id="setting-tray" checked="">
          <span class="slider"></span>
        </label>
      </div>
      <div class="setting-row">
        <div class="setting-info">

```



```

        <strong>Benachrichtigungen</strong>
        <small>Native Windows-Meldungen</small>
    </div>
    <label class="toggle-switch">
        <input type="checkbox" id="setting-notifications" checked>
        <span class="slider"></span>
    </label>
</div>
</div>

<!-- Erscheinungsbild ----- -->
<div class="card settings-card">
    <h3>👤 Erscheinungsbild</h3>
    <div class="setting-row">
        <div class="setting-info">
            <strong>Theme</strong>
        </div>
        <select id="setting-theme" class="setting-select">
            <option value="dark">Dark (Standard)</option>
            <option value="darker">Deep Dark</option>
            <option value="light">Light</option>
        </select>
    </div>
    <div class="setting-row">
        <div class="setting-info">
            <strong>Akzentfarbe</strong>
        </div>
        <div class="color-options">
            <button class="color-opt active" style="background:#7c3aed"
                data-color="#7c3aed"></button>
            <button class="color-opt" style="background:#2563eb"
                data-color="#2563eb"></button>
            <button class="color-opt" style="background:#059669"
                data-color="#059669"></button>
            <button class="color-opt" style="background:#dc2626"
                data-color="#dc2626"></button>
            <button class="color-opt" style="background:#d97706"
                data-color="#d97706"></button>
        </div>
    </div>
</div>

<!-- Server-Defaults ----- -->
<div class="card settings-card">
    <h3>⚡ Server-Defaults</h3>
    <div class="setting-row">
        <div class="setting-info">
            <strong>Default Port</strong>
        </div>
        <input type="number" id="setting-port"
            class="setting-input" value="8080">
    </div>
    <div class="setting-row">
        <div class="setting-info">
            <strong>Default GPU-Layer</strong>
        </div>
        <input type="number" id="setting-ngl"

```

```

        class="setting-input" value="35">
    </div>
    <div class="setting-row">
        <div class="setting-info">
            <strong>Default Kontext</strong>
        </div>
        <input type="number" id="setting-ctx"
            class="setting-input" value="4096">
    </div>
</div>

<!-- Info ----- -->
<div class="card settings-card">
    <h3>📄 Info</h3>
    <div class="info-list">
        <div class="info-row">
            <span>Version</span>
            <span>2.0.0</span>
        </div>
        <div class="info-row">
            <span>Electron</span>
            <span id="info-electron-ver">--</span>
        </div>
        <div class="info-row">
            <span>Node.js</span>
            <span id="info-node-ver">--</span>
        </div>
    </div>
    <div class="info-links">
        <button class="btn btn-sm" id="btn-llama-releases">
            📦 llama.cpp Releases
        </button>
        <button class="btn btn-sm" id="btn-hf-models">
            🤖 HuggingFace Modelle
        </button>
    </div>
</div>

</div>

<!-- Speichern ----- -->
<div class="settings-save-row">
    <button id="btn-save-settings" class="btn btn-start">
        💾 Einstellungen speichern
    </button>
    <span id="settings-saved-hint" class="save-hint"></span>
</div>
</section>

</main>
</div>

<!-- == WIZARD MODAL ----- -->
<div id="wizard-overlay" class="overlay hidden">
    <div class="wizard-modal">
        <button id="close-wizard" class="close-btn">X</button>
        <div id="wizard-content"></div>
    </div>
</div>

```

```
    </div>
  </div>

  <script src="app.js"></script>
</body>
</html>
```



renderer/app.js

() JavaScript

```
// =====
//  Hermes Control Center – Complete Renderer Logic v2
// =====

const api = window.electronAPI

// — App-State —
const state = {
  exePath:    '',
  modelPath:  '',
  port:       8080,
  ngl:        35,
  ctxSize:    4096,
  threads:    8,
  running:    false,
  startTime:  null,
  uptimeTimer: null,
  logEntries: [],
  errorCount: 0,
  accentColor: '#7c3aed',
  currentPage: 'dashboard',
}

// =====
//  INIT
// =====

async function init() {
  await loadSavedSettings()
  await loadLogHistory()
  registerIPCListeners()
  setupNavigation()
  setupLlamaPage()
  setupHermesPanel()
  setupSettingsPage()
  setupLogPage()
  setupWizard()
  setupDashboard()
  setupTitlebar()
  updateAllStatus()
}
```

```

// Versions-Info
document.getElementById('info-electron-ver').textContent =
  navigator.userAgent.match(/ElectronV([\d.]+)/)?.[1] || '?'
document.getElementById('info-node-ver').textContent =
  navigator.userAgent.match(/NodeV([\d.]+)/)?.[1] || '?'
}

// =====
// SETTINGS LADEN
// =====

async function loadSavedSettings() {
  const saved = await api.getSavedPaths()

  if (saved.llamaExePath) updateExePath(saved.llamaExePath)
  if (saved.ggufModelPath) updateModelPath(saved.ggufModelPath)

  state.port      = saved.port      || 8080
  state.ngl       = saved.ngl       || 35
  state.ctxSize   = saved.ctxSize   || 4096

  // Params-Felder befüllen
  setVal('param-port',    state.port)
  setVal('param-ngl',     state.ngl)
  setVal('param-ctx',     state.ctxSize)

  // Settings-Page befüllen
  setVal('setting-port',  state.port)
  setVal('setting-ngl',   state.ngl)
  setVal('setting-ctx',   state.ctxSize)

  // Autostart-Status
  const autostartOn = await api.getAutostartStatus()
  setChecked('setting-autostart', autostartOn)
  setChecked('setting-tray',      saved.minimizeToTray !== false)
  setChecked('setting-notifications', saved.notifications !== false)

  // Theme
  if (saved.theme) {
    document.body.dataset.theme = saved.theme
    setVal('setting-theme', saved.theme)
  }
}

// =====
// IPC LISTENER
// =====

function registerIPCListeners() {
  api.onLogLine(({ level, msg }) => {
    appendLog(level, msg)
    if (level === 'error') {
      state.errorCount++
      updateErrorBadge()
    }
  })
}

api.onServerReady(({ port }) => {
  setServerState(true, port)

```

```

    })

    api.onServerStop(() => {
        setServerState(false)
    })

    api.onTrayStartRequest(() => {
        navigateTo('llama')
    })
}

// =====
// NAVIGATION
// =====

function setupNavigation() {
    document.querySelectorAll('.nav-btn[data-page]').forEach(btn => {
        btn.addEventListener('click', () => navigateTo(btn.dataset.page))
    })

    document.getElementById('nav-quit')?.addEventListener('click', () => {
        // Tray-Hinweis zeigen – echtes Beenden über Tray-Menü
        api.showNotification({
            title: 'Tipp',
            body: 'Benutze das Tray-Menü → "Beenden" zum vollständigen Schließen.'
        })
    })
}

function navigateTo(page) {
    state.currentPage = page

    document.querySelectorAll('.nav-btn').forEach(b =>
        b.classList.toggle('active', b.dataset.page === page)
    )
    document.querySelectorAll('.page').forEach(p =>
        p.classList.toggle('active', p.id === `page-${page}`)
    )

    // Seiten-spezifische Aktionen
    if (page === 'logs') {
        state.errorCount = 0
        updateErrorBadge()
        renderLog()
    }
    if (page === 'hermes' && state.running) {
        checkHermesConnection()
    }
    if (page === 'dashboard') {
        updateDashboard()
    }
}

// =====
// TITLEBAR
// =====

function setupTitlebar() {
    document.getElementById('tb-minimize')?.addEventListener('click', () => {

```

```

// Electron minimieren via IPC wäre ideal – hier als Demo
}))
document.getElementById('tb-tray')?.addEventListener('click', () => {
  api.showNotification({
    title: 'In Tray minimiert',
    body: 'Klicke auf das Hermes-Icon im Tray um die App zu öffnen.'
  })
})
}

// =====
// LLAMA PAGE
// =====
function setupLlamaPage() {
  // Dateiauswahl
  document.getElementById('btn-exe')?.addEventListener('click', async () => {
    const btn = document.getElementById('btn-exe')
    btn.disabled = true
    btn.textContent = '⌛'
    const r = await api.selectLlamaExe()
    btn.disabled = false
    btn.textContent = '📁'
    if (r.success) updateExePath(r.path)
  })

  document.getElementById('btn-model')?.addEventListener('click', async () => {
    const btn = document.getElementById('btn-model')
    btn.disabled = true
    btn.textContent = '⌛'
    const r = await api.selectGgufModel()
    btn.disabled = false
    btn.textContent = '📁'
    if (r.success) updateModelPath(r.path)
  })

  // Im Explorer öffnen
  document.getElementById('btn-exe-open')?.addEventListener('click', () => {
    if (state.exePath) api.openInExplorer(state.exePath)
  })
  document.getElementById('btn-model-open')?.addEventListener('click', () => {
    if (state.modelPath) api.openInExplorer(state.modelPath)
  })

  // Start / Stop
  document.getElementById('btn-start')?.addEventListener('click', startServer)
  document.getElementById('btn-stop')?.addEventListener('click', stopServer)

  // Parameter speichern
  document.getElementById('btn-save-params')?.addEventListener('click', async () => {
    await api.saveSettings({
      port: parseInt(getVal('param-port')),
      ngl: parseInt(getVal('param-ngl')),
      ctxSize: parseInt(getVal('param-ctx')),
      threads: parseInt(getVal('param-threads')) || '8'
    })
    showToast('Parameter gespeichert ✓')
  })
}

```

```

}

function updateExePath(p) {
  state.exePath = p
  setVal('exe-path', p)
  document.getElementById('exe-path').title = p
  const openBtn = document.getElementById('btn-exe-open')
  if (openBtn) openBtn.disabled = !p
  validateStartButton()
}

function updateModelPath(p) {
  state.modelPath = p
  setVal('model-path', p)
  document.getElementById('model-path').title = p
  const openBtn = document.getElementById('btn-model-open')
  if (openBtn) openBtn.disabled = !p
  validateStartButton()
  // Dashboard-Karte aktualisieren
  const modelName = p.split(/[\/\\]/).pop() || '-'
  const el = document.getElementById('dash-model-name')
  if (el) el.textContent = modelName
}

function validateStartButton() {
  const btn = document.getElementById('btn-start')
  if (btn) btn.disabled = state.running || !state.exePath || !state.modelPath
}

// =====
//  SERVER START / STOP
// =====

async function startServer() {
  const config = {
    exePath: state.exePath,
    modelPath: state.modelPath,
    port: parseInt(getVal('param-port')) || 8080,
    ngl: parseInt(getVal('param-ngl')) || 35,
    ctxSize: parseInt(getVal('param-ctx')) || 4096,
    threads: parseInt(getVal('param-threads')) || 8
  }

  state.port = config.port
  appendLog('info', `🚀 Starte llama-server – Port ${config.port}, ngl=${config.ngl}`)

  const r = await api.startLlama(config)
  if (!r.success) {
    appendLog('error', `❌ ${r.error}`)
  }
}

async function stopServer() {
  const r = await api.stopLlama()
  if (!r.success) appendLog('error', `❌ Stop: ${r.error}`)
}

function setServerState(running, port) {

```

```

state.running = running
if (port) state.port = port

// Statusdot überall
const dotClass = running ? 'dot dot-on' : 'dot dot-off'
document.querySelectorAll('#global-dot, #dash-server-dot')
    .forEach(d => d.className = dotClass)

setTextContent('global-status',      running ? 'Läuft' : 'Gestoppt')
setTextContent('dash-server-status', running ? '🟢 Läuft' : '🔴 Gestoppt')

// Start/Stop Buttons
const btnStart = document.getElementById('btn-start')
const btnStop  = document.getElementById('btn-stop')
if (btnStart) btnStart.disabled = running || !state.exePath || !state.modelPath
if (btnStop)  btnStop.disabled  = !running

// Info-Bar
const infoBar = document.getElementById('server-info-bar')
const apiLink = document.getElementById('info-api-link')
const dashApi = document.getElementById('dash-api-url')

if (running) {
    state.startTime = Date.now()
    const url = `http://127.0.0.1:${state.port}`
    if (infoBar) infoBar.classList.remove('hidden')
    if (apiLink) { apiLink.href = url; apiLink.textContent = url }
    if (dashApi) dashApi.textContent = url

    // API-Test-Button aktivieren
    const testBtn = document.getElementById('dash-test-api')
    if (testBtn) { testBtn.disabled = false; testBtn.onclick = checkHermesConnection }

    startUptime()
} else {
    if (infoBar) infoBar.classList.add('hidden')
    if (dashApi) dashApi.textContent = '-'
    const testBtn = document.getElementById('dash-test-api')
    if (testBtn) testBtn.disabled = true
    stopUptime()
}

updateDashboard()
}

// — Uptime-Counter —————
function startUptime() {
    stopUptime()
    state.uptimeTimer = setInterval(() => {
        const s = Math.floor((Date.now() - state.startTime) / 1000)
        const h = Math.floor(s / 3600)
        const m = Math.floor((s % 3600) / 60)
        const sec = s % 60
        const txt = `${h > 0 ? h+'h ' : ''}${m > 0 ? m+'m ' : ''}${sec}s`
        setTextContent('info-uptime', `🕒 ${txt}`)
        setTextContent('dash-uptime', txt)
    }, 1000)
}

```



```

}

function stopUptime() {
  if (state.uptimeTimer) {
    clearInterval(state.uptimeTimer)
    state.uptimeTimer = null
  }
  settextContent('dash-uptime', '-')
}

// =====
// DASHBOARD
// =====

function setupDashboard() {
  document.getElementById('qa-start')?.addEventListener('click', () => {
    navigateTo('llama')
    if (!state.running && state.exePath && state.modelPath) startServer()
  })
  document.getElementById('qa-wizard')?.addEventListener('click', openWizard)
  document.getElementById('qa-api')?.addEventListener('click', () => {
    if (state.running) api.openExternalUrl(`http://127.0.0.1:${state.port}`)
  })
  document.getElementById('qa-logs')?.addEventListener('click',
    () => navigateTo('logs')
  )
}

function updateDashboard() {
  const modelPath = state.modelPath
  const modelName = modelPath ? modelPath.split(/[\/\\]/).pop() : 'Nicht geladen'
  settextContent('dash-model-name', modelName)
}

function updateAllStatus() {
  updateDashboard()
}

// =====
// HERMES PANEL
// =====

function setupHermesPanel() {
  document.getElementById('btn-hermes-connect')?.addEventListener('click',
    checkHermesConnection
  )
  document.getElementById('btn-hermes-refresh')?.addEventListener('click',
    checkHermesConnection
  )
  document.getElementById('btn-api-test')?.addEventListener('click',
    runApiTest
  )
}

async function checkHermesConnection() {
  const dot = document.getElementById('hermes-dot')
  const text = document.getElementById('hermes-status-text')
  const details = document.getElementById('hermes-details')

```

```

if (dot) dot.className = 'dot dot-off'
if (text) text.textContent = 'Verbinde...'

const result = await api.apiHealthCheck(state.port)

if (result.success) {
  if (dot) dot.className = 'dot dot-on'
  if (text) text.textContent = 'Verbunden ✓'
  if (details) details.style.display = 'block'

  const ep = document.getElementById('hermes-endpoint')
  const hl = document.getElementById('hermes-health')
  const ml = document.getElementById('hermes-model')

  if (ep) ep.textContent = `http://127.0.0.1:${state.port}`
  if (hl) hl.textContent = result.status?.status || 'ok'
  if (ml) ml.textContent = state.modelPath?.split(/[\\\/]/).pop() || '-'

  await loadModelInfo()
} else {
  if (dot) dot.className = 'dot dot-off'
  if (text) text.textContent = `Fehler: ${result.error || 'Nicht erreichbar'}`
  if (details) details.style.display = 'none'
}
}

async function loadModelInfo() {
  const container = document.getElementById('model-info-content')
  if (!container) return

  try {
    const resp = await fetch(`http://127.0.0.1:${state.port}/v1/models`)
    const data = await resp.json()
    const models = data.data || []

    if (models.length > 0) {
      container.innerHTML = models.map(m => `
        <div class="model-info-row">
          <div class="mir-name">${m.id} || '-'</div>
          <div class="mir-meta">
            <span>Erstellt: ${m.created
              ? new Date(m.created * 1000).toLocaleDateString('de-DE')
              : '-'</span>
            <span>Eigentümer: ${m.owned_by} || '-'</span>
          </div>
        </div>
      `).join('')
    } else {
      container.innerHTML = '<div class="info-placeholder">Keine Modelle gefunden.</div>'
    }
  } catch {
    container.innerHTML = '<div class="info-placeholder">Modell-Info nicht abrufbar.</div>'
  }
}

async function runApiTest() {
  const prompt = document.getElementById('api-test-prompt')?.value?.trim()

```

```

const respBox = document.getElementById('api-response-box')
const stats = document.getElementById('api-stats')
const btn = document.getElementById('btn-api-test')

if (!prompt) return
if (!state.running) {
  if (respBox) respBox.innerHTML = '<div class="response-error">Server ist nicht gestartet!</div>'
  return
}

if (btn) { btn.disabled = true; btn.textContent = '⌚ Sende...' }
if (respBox) respBox.innerHTML = '<div class="response-loading">⌚ Warte auf Antwort...</div>'
if (stats) stats.style.display = 'none'

const startTs = Date.now()

try {
  const resp = await fetch(`http://127.0.0.1:${state.port}/v1/chat/completions`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'hermes',
      messages: [{ role: 'user', content: prompt }],
      max_tokens: 512,
      temperature: 0.7
    })
  })

  const data = await resp.json()
  const elapsed = Date.now() - startTs
  const answer = data.choices?.[0]?.message?.content || '(keine Antwort)'
  const tokens = data.usage?.completion_tokens || '?'

  if (respBox) {
    respBox.innerHTML = `<div class="response-text">${escapeHtml(answer)}</div>`
  }

  if (stats) {
    stats.style.display = 'flex'
    setTextContent('stat-time', `${elapsed}ms`)
    setTextContent('stat-tokens', `${tokens} tokens`)
  }

} catch (err) {
  if (respBox) respBox.innerHTML =
    `<div class="response-error">Fehler: ${escapeHtml(err.message)}</div>`
} finally {
  if (btn) { btn.disabled = false; btn.textContent = '▶ Test senden' }
}

// =====
// SETTINGS PAGE
// =====

function setupSettingsPage() {
  // Akzentfarben
  document.querySelectorAll('.color-opt').forEach(btn => {

```

```

    btn.addEventListener('click', () => {
      document.querySelectorAll('.color-opt')
        .forEach(b => b.classList.remove('active'))
      btn.classList.add('active')
      state.accentColor = btn.dataset.color
      document.documentElement.style.setProperty('--accent', state.accentColor)
    })
  })

  // Theme
  document.getElementById('setting-theme')?.addEventListener('change', (e) => {
    document.body.dataset.theme = e.target.value
  })

  // Speichern
  document.getElementById('btn-save-settings')?.addEventListener('click',
    saveSettings
  )

  // Links
  document.getElementById('btn-llama-releases')?.addEventListener('click', () =>
    api.openExternalUrl('https://github.com/ggml-org/llama.cpp/releases')
  )
  document.getElementById('btn-hf-models')?.addEventListener('click', () =>
    api.openExternalUrl('https://huggingface.co/models?library=gguf')
  )
}

async function saveSettings() {
  const settings = {
    port:          parseInt(getVal('setting-port')) || 8080,
    ngl:           parseInt(getVal('setting-ngl')) || 35,
    ctxSize:       parseInt(getVal('setting-ctx')) || 4096,
    autostart:     isChecked('setting-autostart'),
    minimizeToTray: isChecked('setting-tray'),
    notifications: isChecked('setting-notifications'),
    theme:         getVal('setting-theme') || 'dark',
    accentColor:   state.accentColor
  }

  await api.saveSettings(settings)

  // Params-Seite synchronisieren
  setVal('param-port', settings.port)
  setVal('param-ngl', settings.ngl)
  setVal('param-ctx', settings.ctxSize)

  const hint = document.getElementById('settings-saved-hint')
  if (hint) {
    hint.textContent = '✓ Gespeichert!'
    hint.style.color = 'var(--green)'
    setTimeout(() => hint.textContent = '', 2500)
  }
}

// =====
// LOG PAGE

```

```
// =====
async function loadLogHistory() {
  const history = await api.getLogHistory()
  history.forEach(e => appendLog(e.level, e.msg, false))
}

function setupLogPage() {
  document.getElementById('log-search')?.addEventListener('input', renderLog)
  document.getElementById('filter-errors')?.addEventListener('change', renderLog)
  document.getElementById('btn-clear-log')?.addEventListener('click', () => {
    state.logEntries = []
    state.errorCount = 0
    updateErrorBadge()
    const viewer = document.getElementById('log-viewer')
    if (viewer) viewer.innerHTML =
      '<div class="log-placeholder">Log geleert.</div>'
    setTextContent('log-count', '0 Einträge')
  })
  document.getElementById('btn-export-log')?.addEventListener('click', exportLog)
}

function appendLog(level, msg, render = true) {
  const entry = {
    ts:    new Date().toLocaleTimeString('de-DE'),
    level: level || 'info',
    msg:   (msg || '').trim()
  }
  if (!entry.msg) return
  state.logEntries.push(entry)
  if (state.logEntries.length > 5000) state.logEntries.shift()
  if (render) renderLog()
}

function renderLog() {
  const viewer = document.getElementById('log-viewer')
  if (!viewer) return

  const query = document.getElementById('log-search')?.value?.toLowerCase() || ''
  const errorsOnly = document.getElementById('filter-errors')?.checked || false
  const autoScroll = document.getElementById('filter-scroll')?.checked !== false

  const filtered = state.logEntries.filter(e => {
    if (errorsOnly && e.level === 'info') return false
    if (query && !e.msg.toLowerCase().includes(query)) return false
    return true
  })

  // Immer komplett neu rendern (korrekt bei Suche/Filter)
  const frag = document.createDocumentFragment()

  if (filtered.length === 0) {
    const ph = document.createElement('div')
    ph.className = 'log-placeholder'
    ph.textContent = 'Keine Einträge.'
    frag.appendChild(ph)
  } else {
    filtered.forEach(e => frag.appendChild(createLogLine(e, query)))
  }
}

```

```

}

viewer.innerHTML = ''
viewer.appendChild(frag)

setTextContent('log-count',
  filtered.length !== state.logEntries.length
    ? `${filtered.length} / ${state.logEntries.length} Einträge`
    : `${state.logEntries.length} Einträge`
)

if (autoScroll) viewer.scrollTop = viewer.scrollHeight
}

function createLogline(entry, query = '') {
  const line = document.createElement('div')
  line.className = `log-line log-${entry.level}`

  const ts = document.createElement('span')
  ts.className = 'log-ts'
  ts.textContent = entry.ts

  const lvl = document.createElement('span')
  lvl.className = 'log-lvl'
  lvl.textContent = entry.level.toUpperCase().padEnd(5)

  const msg = document.createElement('span')
  msg.className = 'log-msg'

  if (query) {
    const re = new RegExp(`(${escapeRegex(query)})`, 'gi')
    msg.innerHTML = escapeHtml(entry.msg).replace(re, '<mark>$1</mark>')
  } else {
    msg.textContent = entry.msg
  }

  line.appendChild(ts)
  line.appendChild(lvl)
  line.appendChild(msg)
  return line
}

function updateErrorBadge() {
  const badge = document.getElementById('error-badge')
  if (!badge) return
  if (state.errorCount > 0 && state.currentPage !== 'logs') {
    badge.style.display = 'block'
    badge.textContent = state.errorCount > 99 ? '99+' : String(state.errorCount)
  } else {
    badge.style.display = 'none'
  }
}

function exportLog() {
  const text = state.logEntries
    .map(e => `${e.ts} [${e.level.toUpperCase()}] ${e.msg}`)
    .join('\n')

```

```

const blob = new Blob([text], { type: 'text/plain' })
const url = URL.createObjectURL(blob)
const a = document.createElement('a')
a.href = url
a.download = `hermes-log-${Date.now()}.txt`
a.click()
URL.revokeObjectURL(url)
}

// =====
//  SETUP WIZARD
//  =====

const WIZARD_STEPS = [
  {
    id: 'welcome', title: '👋 Willkommen',
    html: `
      <p>Dieser Assistent führt dich in <strong>4 Schritten</strong> durch die Einrichtung.</p>
      <ul>
        <li>✅ llama-server.exe finden</li>
        <li>✅ GGUF-Modell auswählen</li>
        <li>✅ Parameter optimieren</li>
        <li>✅ Ersten Start durchführen</li>
      </ul>
      <div class="hint-box">
        💡 Du brauchst llama.cpp – aktuelle Releases auf GitHub unter
        <strong>ggml-org/llama.cpp</strong>
      </div>
    `,
  },
  {
    id: 'exe', title: '📁 llama-server.exe',
    html: `
      <p>Wähle die <code>llama-server.exe</code> aus deiner llama.cpp-Installation.</p>
      <div class="hint-box">📁 Typischer Pfad: <code>llama.cpp\\build\\bin\\Release\\</code></div>
      <div id="w-exe-status" class="wizard-status-line"></div>
    `,
    action: async () => {
      const r = await api.selectLlamaExe()
      if (r.success) {
        updateExePath(r.path)
        document.getElementById('w-exe-status').innerHTML =
          `<span class="ok">✅ ${r.path}</span>`
      }
    }
  },
  {
    id: 'model', title: '🧠 GGUF-Modell',
    html: `
      <p>Wähle dein <code>.gguf</code>-Modell aus.</p>
      <div class="hint-box">
        Empfohlene Modelle:<br>
        • <strong>Hermes-3-Llama-3.2-3B.Q4_K_M.gguf</strong> (~2GB, sehr schnell)<br>
        • <strong>Mistral-7B-Instruct-v0.3.Q4_K_M.gguf</strong> (~4GB)<br>
        • <strong>Llama-3.1-8B-Instruct-Q4_K_M.gguf</strong> (~5GB)
      </div>
      <div id="w-model-status" class="wizard-status-line"></div>
    `,
  },

```



```

    <div class="sum-row"> 🍷 <b>Modell:</b> ${state.modelPath || '-'}</div>
    <div class="sum-row"> ⚡ <b>GPU-Layer:</b> ${getVal('param-ngl')}</div>
    <div class="sum-row"> 📄 <b>Kontext:</b> ${getVal('param-ctx')}</div>
    <div class="sum-row"> 🌐 <b>Port:</b> ${getVal('param-port')}</div>
  },
}
]

let wizardStep = 0
window.wSetNgl = v => { const el = document.getElementById('w-ngl'); if (el) el.value = v }
window.wSetCtx = v => { const el = document.getElementById('w-ctx'); if (el) el.value = v }

function setupWizard() {
  document.getElementById('close-wizard')?.addEventListener('click', closeWizard)
  document.getElementById('wizard-overlay')?.addEventListener('click', e => {
    if (e.target.id === 'wizard-overlay') closeWizard()
  })
  document.addEventListener('keydown', e => { if (e.key === 'Escape') closeWizard() })

  // Globale Wizard-Funktionen
  window.wizardNext = wizardNextFn
  window.wizardBack = wizardBackFn
  window.wizardAction = wizardActionFn
  window.wizardFinish = wizardFinishFn
}

function openWizard() {
  wizardStep = 0
  renderWizard()
  document.getElementById('wizard-overlay')?.classList.remove('hidden')
}

function closeWizard() {
  document.getElementById('wizard-overlay')?.classList.add('hidden')
}

function renderWizard() {
  const step = WIZARD_STEPS[wizardStep]
  const total = WIZARD_STEPS.length
  const cont = document.getElementById('wizard-content')
  if (!cont) return

  cont.innerHTML = `
    <div class="wiz-progress">
      ${WIZARD_STEPS.map((s, i) => `
        <div class="wp-step ${i === wizardStep ? 'active' : i < wizardStep ? 'done' : ''}">
          <div class="wp-dot">${i < wizardStep ? '✓' : i + 1}</div>
          <div class="wp-lbl">${s.id}</div>
        </div>
        ${i < total - 1 ? '<div class="wp-line"></div>' : ''}
      `).join('')}
    </div>
    <div class="wiz-body">
      <h3>${step.title}</h3>
      <div class="wiz-text">${step.html}</div>
    </div>
    <div class="wiz-footer">

```

```

<button class="btn" onclick="wizardBack()"
    ${wizardStep === 0 ? 'disabled' : ''}>< Zurück</button>
${step.action ? `<button class="btn btn-pick"
    onclick="wizardAction()">📁 Auswählen</button>` : ''}
${wizardStep < total - 1
    ? `<button class="btn btn-start" onclick="wizardNext()">Weiter →</button>`
    : `<button class="btn btn-start" onclick="wizardFinish()">🚀 Starten!</button>`}
}
</div>
,
step.onShow?().()
}

async function wizardActionFn() {
    await WIZARD_STEPS[wizardStep]?.action?().()
}
function wizardNextFn() {
    if (wizardStep < WIZARD_STEPS.length - 1) { wizardStep++; renderWizard() }
}
function wizardBackFn() {
    if (wizardStep > 0) { wizardStep--; renderWizard() }
}
async function wizardFinishFn() {
    closeWizard()
    navigateTo('llama')
    if (state.exePath && state.modelPath) await startServer()
}

// =====
// TOAST
// =====
function showToast(msg, duration = 2500) {
    let toast = document.getElementById('toast')
    if (!toast) {
        toast = document.createElement('div')
        toast.id = 'toast'
        toast.style.cssText = `
            position:fixed; bottom:20px; right:20px; z-index:9999;
            background:var(--surface2); border:1px solid var(--border);
            color:var(--text); padding:10px 18px; border-radius:8px;
            font-size:0.85rem; box-shadow:0 4px 20px rgba(0,0,0,0.4);
            transition:opacity 0.3s; opacity:0;
        `
        document.body.appendChild(toast)
    }
    toast.textContent = msg
    toast.style.opacity = '1'
    clearTimeout(toast._timer)
    toast._timer = setTimeout(() => toast.style.opacity = '0', duration)
}

// =====
// UTILS
// =====
const getVal = id => document.getElementById(id)?.value ?? ''
const setVal = (id, v) => { const el = document.getElementById(id); if (el) el.value = v }
const isChecked= id => document.getElementById(id)?.checked ?? false

```

```
const setChecked=(id,v)=>{ const el = document.getElementById(id); if (el) el.checked = v }
const settextContent = (id, t) => { const el = document.getElementById(id); if (el) el.textContent = t }
const escapeHtml = s => s.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;')
const escapeRegex= s => s.replace(/[\.\*\+\?\^\$\{\}\(\)\[\]\\\]/g, '\\$&')

// =====
//  START
// =====
init()
```

5 styles.css

{ } CSS

```
/* =====
   Ergänzungen zum bestehenden styles.css
   (Sidebar, Pages, Settings, Hermes Panel, Wizard)
   ===== */

/* — Layout ————— */
.app-layout {
  display: flex;
  flex: 1;
  overflow: hidden;
}

/* — Sidebar ————— */
.sidebar {
  width: 180px;
  background: var(--surface);
  border-right: 1px solid var(--border);
  display: flex;
  flex-direction: column;
  padding: 8px 0;
  flex-shrink: 0;
}

.nav-btn {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 10px 16px;
  background: transparent;
  border: none;
  color: var(--text-dim);
  cursor: pointer;
  font-size: 0.85rem;
  width: 100%;
  text-align: left;
  border-radius: 0;
  transition: all 0.15s;
```

```

    position: relative;
}

.nav-btn:hover { background: var(--surface2); color: var(--text); }
.nav-btn.active {
    background: rgba(124, 58, 237, 0.15);
    color: var(--accent);
    border-left: 3px solid var(--accent);
}

.nav-icon { font-size: 1rem; width: 20px; text-align: center; }
.nav-badge {
    position: absolute;
    right: 10px;
    top: 50%;
    transform: translateY(-50%);
    background: var(--red);
    color: white;
    border-radius: 10px;
    padding: 1px 6px;
    font-size: 0.7rem;
    font-weight: 700;
}

.sidebar-bottom { margin-top: auto; }
.nav-btn-danger:hover { color: var(--red) !important; }

/* — Page Container — */
.page-container {
    flex: 1;
    overflow-y: auto;
    background: var(--bg);
}

.page {
    display: none;
    padding: 24px;
    min-height: 100%;
    animation: fadeIn 0.15s ease;
}

.page.active { display: block; }

@keyframes fadeIn {
    from { opacity: 0; transform: translateY(4px); }
    to { opacity: 1; transform: translateY(0); }
}

.page-header {
    margin-bottom: 24px;
}

.page-header h1 {
    font-size: 1.3rem;
    font-weight: 700;
    margin-bottom: 4px;
}

.page-subtitle { color: var(--text-dim); font-size: 0.85rem; }

```

```

/* — Status Cards (Dashboard) ————— */
.status-cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 12px;
  margin-bottom: 24px;
}

.status-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 16px;
  display: flex;
  align-items: center;
  gap: 14px;
  transition: border-color 0.2s;
}

.status-card:hover { border-color: var(--accent); }

.sc-icon { font-size: 1.6rem; }
.sc-body { flex: 1; }
.sc-label { font-size: 0.75rem; color: var(--text-dim); text-transform: uppercase; }
.sc-value { font-size: 0.95rem; font-weight: 600; margin-top: 2px; }
.sc-action {
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text-dim);
  padding: 5px 10px;
  border-radius: 6px;
  cursor: pointer;
  font-size: 0.78rem;
  transition: all 0.15s;
}

.sc-action:hover:not(:disabled) { color: var(--text); border-color: var(--accent); }
.sc-action:disabled { opacity: 0.4; cursor: not-allowed; }

/* — Quick Actions ————— */
.quick-actions h2 {
  font-size: 0.9rem;
  color: var(--text-dim);
  text-transform: uppercase;
  letter-spacing: 1px;
  margin-bottom: 12px;
}

.qa-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 10px;
}

.qa-btn {
  display: flex;
  align-items: center;
  gap: 14px;
  background: var(--surface);

```

```

border: 1px solid var(--border);
color: var(--text);
padding: 14px 16px;
border-radius: 10px;
cursor: pointer;
text-align: left;
transition: all 0.15s;
}

.qa-btn:hover { border-color: var(--accent); background: var(--surface2); }
.qa-btn > span { font-size: 1.4rem; }
.qa-btn strong { display: block; font-size: 0.9rem; }
.qa-btn small { color: var(--text-dim); font-size: 0.75rem; }

/* — Two-Col Layout — */
.two-col {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}

/* — Card — */
.card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 18px;
}

.card h3 {
  font-size: 0.9rem;
  color: var(--text-dim);
  text-transform: uppercase;
  letter-spacing: 0.5px;
  margin-bottom: 14px;
}

/* — Info Bar — */
.info-bar {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-top: 14px;
  padding: 10px 14px;
  background: rgba(34, 197, 94, 0.08);
  border: 1px solid var(--green);
  border-radius: 8px;
  font-size: 0.82rem;
}

.info-bar a { color: var(--accent); text-decoration: none; }

/* — Hermes Panel — */
.hermes-panel {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: auto auto;
  gap: 16px;
}

```

```

.connect-card { grid-column: 1; }
.api-test-card { grid-column: 2; }
.model-info-card { grid-column: 1 / -1; }

.connect-status { display: flex; flex-direction: column; gap: 12px; }
.cs-indicator { display: flex; align-items: center; gap: 10px; font-size: 0.9rem; }

.cs-details { background: var(--surface2); border-radius: 8px; padding: 12px; }
.detail-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 4px 0;
  font-size: 0.82rem;
  border-bottom: 1px solid var(--border);
}
.detail-row:last-child { border: none; }
.detail-row code {
  background: var(--bg);
  padding: 2px 6px;
  border-radius: 4px;
  font-size: 0.78rem;
}

.cs-actions { display: flex; gap: 8px; }

.api-test { display: flex; flex-direction: column; gap: 10px; }

.api-test textarea {
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 8px 10px;
  border-radius: 6px;
  font-size: 0.85rem;
  resize: vertical;
  font-family: inherit;
}

.api-response-box {
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 6px;
  padding: 12px;
  min-height: 80px;
  font-size: 0.82rem;
  line-height: 1.6;
}

.response-placeholder { color: var(--text-dim); font-style: italic; }
.response-loading { color: var(--yellow); }
.response-error { color: var(--red); }
.response-text { color: var(--text); white-space: pre-wrap; word-break: break-word; }

.api-stats {
  display: flex;

```

```

    gap: 16px;
    font-size: 0.78rem;
    color: var(--text-dim);
}

.model-info { padding: 4px 0; }
.model-info-row {
    padding: 8px;
    border-bottom: 1px solid var(--border);
}

.mir-name { font-weight: 600; font-size: 0.9rem; font-family: var(--font-mono); }
.mir-meta {
    display: flex;
    gap: 16px;
    font-size: 0.78rem;
    color: var(--text-dim);
    margin-top: 4px;
}

.info-placeholder { color: var(--text-dim); font-style: italic; padding: 12px; }

/* — Settings — */
.settings-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
    gap: 16px;
    margin-bottom: 20px;
}

.settings-card { /* inherits .card */ }

.setting-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 10px 0;
    border-bottom: 1px solid var(--border);
}

.setting-row:last-child { border: none; }

.setting-info strong { display: block; font-size: 0.88rem; }
.setting-info small { color: var(--text-dim); font-size: 0.75rem; }

/* Toggle Switch */
.toggle-switch { position: relative; width: 40px; height: 22px; }
.toggle-switch input { opacity: 0; width: 0; height: 0; }
.slider {
    position: absolute; inset: 0;
    background: var(--border);
    border-radius: 22px;
    cursor: pointer;
    transition: 0.2s;
}

.slider:before {
    content: '';
    position: absolute;
    width: 16px; height: 16px;
    left: 3px; top: 3px;

```



```

    background: white;
    border-radius: 50%;
    transition: 0.2s;
}
.toggle-switch input:checked + .slider { background: var(--accent); }
.toggle-switch input:checked + .slider:before { transform: translateX(18px); }

/* Setting-Input */
.setting-input {
    background: var(--surface2);
    border: 1px solid var(--border);
    color: var(--text);
    padding: 5px 8px;
    border-radius: 6px;
    width: 90px;
    font-size: 0.85rem;
}

.setting-select {
    background: var(--surface2);
    border: 1px solid var(--border);
    color: var(--text);
    padding: 5px 8px;
    border-radius: 6px;
    font-size: 0.85rem;
}

/* Akzentfarben */
.color-options { display: flex; gap: 6px; }
.color-opt {
    width: 22px; height: 22px;
    border-radius: 50%;
    border: 2px solid transparent;
    cursor: pointer;
    transition: border-color 0.15s;
}
.color-opt.active { border-color: white; }
.color-opt:hover { transform: scale(1.1); }

/* Info-List */
.info-list { margin-bottom: 14px; }
.info-row {
    display: flex;
    justify-content: space-between;
    padding: 6px 0;
    font-size: 0.82rem;
    border-bottom: 1px solid var(--border);
    color: var(--text-dim);
}
.info-links { display: flex; gap: 8px; flex-wrap: wrap; }

.settings-save-row {
    display: flex;
    align-items: center;
    gap: 14px;
}
.save-hint { font-size: 0.85rem; }

```

```
/* — Log Page ————— */
.log-toolbar {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}
.log-search-input {
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 5px 10px;
  border-radius: 6px;
  font-size: 0.82rem;
  width: 180px;
}
.toggle-label {
  display: flex;
  align-items: center;
  gap: 5px;
  font-size: 0.8rem;
  color: var(--text-dim);
  cursor: pointer;
}

.log-viewer-full {
  flex: 1;
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow-y: auto;
  min-height: calc(100vh - 220px);
  font-family: var(--font-mono);
  font-size: 0.78rem;
}

.log-footer {
  display: flex;
  gap: 16px;
  margin-top: 8px;
  font-size: 0.75rem;
  color: var(--text-dim);
}

/* — Wizard Erweiterungen ————— */
.hint-box {
  background: var(--surface2);
  border-left: 3px solid var(--accent);
  padding: 10px 14px;
  border-radius: 0 6px 6px 0;
  font-size: 0.82rem;
  color: var(--text-dim);
  margin: 10px 0;
  line-height: 1.6;
}
```

```

.wizard-presets { display: flex; flex-direction: column; gap: 20px; }
.preset-group h4 { margin-bottom: 8px; font-size: 0.9rem; }
.preset-btns { display: flex; gap: 8px; flex-wrap: wrap; margin-bottom: 8px; }
.preset {
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text-dim);
  padding: 8px 14px;
  border-radius: 8px;
  cursor: pointer;
  font-size: 0.8rem;
  text-align: center;
  line-height: 1.5;
  transition: all 0.15s;
}
.preset:hover { border-color: var(--accent); color: var(--text); }
.preset.active { background: var(--accent); border-color: var(--accent); color: white; }
.preset code { font-size: 0.7rem; color: inherit; opacity: 0.8; }

.wizard-status-line { margin-top: 10px; min-height: 22px; font-size: 0.85rem; }
.wizard-status-line .ok { color: var(--green); }

.wizard-summary {
  background: var(--surface2);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 14px;
  margin: 14px auto;
  text-align: left;
  max-width: 400px;
}
.sum-row {
  padding: 5px 0;
  font-size: 0.83rem;
  border-bottom: 1px solid var(--border);
  word-break: break-all;
  color: var(--text-dim);
}
.sum-row:last-child { border: none; }

/* Wizard Footer */
.wiz-footer {
  display: flex;
  gap: 8px;
  justify-content: space-between;
  margin-top: 20px;
  padding-top: 16px;
  border-top: 1px solid var(--border);
}

/* Wizard Progress */
.wiz-progress {
  display: flex;
  align-items: center;
  justify-content: center;
  margin-bottom: 22px;
}

```

```

.wp-step {
  display: flex; flex-direction: column; align-items: center; gap: 3px;
}
.wp-dot {
  width: 26px; height: 26px; border-radius: 50%;
  background: var(--surface2); border: 2px solid var(--border);
  display: flex; align-items: center; justify-content: center;
  font-size: 0.75rem; font-weight: 700; color: var(--text-dim);
  transition: all 0.2s;
}
.wp-step.active .wp-dot { background: var(--accent); border-color: var(--accent); color: white; }
.wp-step.done .wp-dot { background: var(--green); border-color: var(--green); color: white; }
.wp-lbl { font-size: 0.6rem; color: var(--text-dim); text-transform: uppercase; }
.wp-line { flex: 1; height: 2px; background: var(--border); min-width: 15px; margin-bottom: 14px; }

/* Titlebar */
.titlebar {
  height: 42px;
  background: var(--surface);
  border-bottom: 1px solid var(--border);
  display: flex;
  align-items: center;
  padding: 0 14px;
  gap: 10px;
  -webkit-app-region: drag;
  user-select: none;
  flex-shrink: 0;
}
.titlebar-logo { font-size: 1.1rem; }
.titlebar-title { font-weight: 600; font-size: 0.9rem; color: var(--accent); }
.titlebar-status {
  margin-left: auto;
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 0.8rem;
  color: var(--text-dim);
}
.titlebar-actions { display: flex; gap: 6px; -webkit-app-region: no-drag; }
.tb-btn {
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text-dim);
  width: 26px; height: 26px;
  border-radius: 5px;
  cursor: pointer;
  font-size: 0.8rem;
  transition: all 0.15s;
}
.tb-btn:hover { color: var(--text); border-color: var(--accent); }

/* Responsive */
@media (max-width: 900px) {
  .two-col { grid-template-columns: 1fr; }
  .hermes-panel { grid-template-columns: 1fr; }
  .model-info-card { grid-column: 1; }
  .sidebar { width: 50px; }
}

```

```
.nav-label      { display: none; }  
}
```



docs/GOOGLE_AI_STUDIO_ÜBERGABE_PROMPT_V2.md

[] Markdown

```
# HERMES CONTROL CENTER – KI-ÜBERGABE-PROMPT V2  
# Für: Google AI Studio, Claude, GPT-4, Gemini Pro, Copilot  
  
---  
  
## 🚀 AUFGABE  
  
Du übernimmst die Weiterentwicklung des "Hermes Control Centers",  
einer vollständigen Electron-Desktop-Anwendung für Windows.  
  
Die App ist kein Web-Prototyp. Sie ist eine echte Desktop-App:  
- Electron (Haupt-Prozess + Renderer + Preload, contextIsolation=true)  
- Keine Frameworks im Renderer (reines JS, HTML, CSS)  
- Backend optional (Node.js/Express für API-Proxy)  
- Zielplattform: Windows 10/11 x64  
  
---  
  
## 🏠 BESTEHENDER CODE-STAND  
  
### Dateien die existieren:
```

main.js ← Electron Main Process preload.js ← contextBridge / IPC renderer/ index.html ← Haupt-UI (Multi-Page mit Sidebar-Navigation) app.js ← Komplette UI-Logik (~600 Zeilen) styles.css ← Dark Theme, vollständig

[] text

```
### Was bereits funktioniert:  
1. ✅ Native Dateiauswahl (dialog.showOpenDialog) für .exe und .gguf  
2. ✅ llama-server starten/stoppen (child_process.spawn)  
3. ✅ Echtzeit-Log-Viewer (stdout/stderr → IPC → Renderer)  
4. ✅ Log-Suche, Filter, Export  
5. ✅ Persistenz aller Einstellungen (electron-store)  
6. ✅ Tray-Icon mit Kontextmenü  
7. ✅ Autostart via app.setLoginItemSettings  
8. ✅ Native Windows-Benachrichtigungen (Notification API)  
9. ✅ Hermes Quick Connect Panel mit API-Test
```

10.  5-Schritte Setup-Assistent (Wizard)
11.  Settings-Page mit Toggle-Switches
12.  Dashboard mit Status-Cards
13.  Multi-Page SPA mit Sidebar-Navigation

🍷 WAS DAS PRODUKT IST

****Hermes Control Center**** ist ein Desktop-Launcher und Kontrollzentrum für lokale KI-Modelle auf Basis von llama.cpp.

Kernkomponenten:

1. ****llama-server.exe**** (aus llama.cpp) – der lokale KI-Inferenz-Server
2. ****GGUF-Modelle**** – quantisierte Sprachmodelle (Hermes, Mistral, Llama)
3. ****OpenAI-kompatible API**** – läuft auf `http://127.0.0.1:8080/v1`

Zielgruppe:

- Technisch versierte Windows-Nutzer
- Entwickler die lokal mit LLMs arbeiten
- Datenschutzbewusste Nutzer (100% offline)

Was das Produkt NICHT ist:

- Kein Cloud-Dienst
- Kein Web-App (auch wenn es wie eine aussieht)
- Kein ChatGPT-Klon
- Keine Electron-Wrapper um eine Website

🛠️ TECHNISCHE REGELN – STRIKT EINHALTEN

Sicherheit:

- `contextIsolation: true` → NIEMALS auf `false` setzen
- `nodeIntegration: false` → NIEMALS aktivieren
- Alle IPC-Kanäle: NUR über `contextBridge.exposeInMainWorld()`
- Keine `remote-module` Nutzung
- Kein `require()` im Renderer

IPC-Pattern (PFLICHT):

```
```javascript
// Main: ipcMain.handle('kanalname', async (_, params) => { ... })
// Preload: ipcRenderer.invoke('kanalname', params)
// UI: await window.electronAPI.methodenName(params)
```

- `app.setLoginItemSettings()` für Autostart (kein VBScript, kein Registry-Hack)
- `dialog.showOpenDialog()` für Dateiauswahl (kein `input[type=file]`)
- `shell.showItemInFolder()` für Explorer-Integration
- Notification API für native Benachrichtigungen
- Tray: Referenz MUSS global gehalten werden (sonst GC)



### 1. Auto-Updater — electron-updater + GitHub Releases

- Datei: main.js erweitern
- NPM: electron-updater installieren

### 2. Hugging Face Modell-Browser

- Neue Seite: page-models
- HF Search API: <https://huggingface.co/api/models?filter=gguf>
- Download mit Fortschrittsanzeige via fetch + ReadableStream

### 3. Chat-Interface

- Neue Seite: page-chat
- OpenAI-kompatible fetch-Calls an 127.0.0.1:PORT/v1/chat/completions
- Streaming via EventSource oder ReadableStream
- Markdown-Rendering (marked.js, kein CDN → lokal bundlen)

### 4. Mehrere Modell-Profile

- electron-store: profiles[] Array
- Schnelles Wechseln zwischen Konfigurationen

### 5. GPU-Diagnose

- nvidia-smi.exe aufrufen (spawn)
- VRAM-Anzeige im Dashboard

### 6. Server-Prozess-Monitor

- RAM-Verbrauch des llama-server Prozesses
- CPU-Last-Anzeige
- Node.js: process.memoryUsage() + os.cpus()



- Reines vanilla JS (kein React/Vue im Renderer)
  - Arrow Functions überall
  - Async/Await statt .then()
  - DOM-Zugriff immer mit optional chaining (?.)
  - Keine globalen Variablen außer `state` und `api`
  - Funktionen: camelCase, beschreibende Namen
  - CSS: CSS Custom Properties (--var-name)
  - Keine externen CDN-Links (CSP blockiert sie)
- 



`{}` text

- ✗ `webContents.executeJavaScript()` aus dem Main-Prozess
  - ✗ `eval()` oder `innerHTML` mit unkontrollierten Daten
  - ✗ `require('electron')` im Renderer-Prozess
  - ✗ `nodeIntegration: true`
  - ✗ `contextIsolation: false`
  - ✗ `remote module` (deprecated seit Electron 14)
  - ✗ CDN-URLs in HTML (CSP-Konflikt)
  - ✗ Synchroner IPC (`ipcRenderer.sendSync`)
  - ✗ `app.quit()` in `window-all-closed` wenn Tray aktiv
- 



1. Zeige immer welche Datei du änderst: `// === DATEI: main.js ===`
  2. Zeige vollständige Funktionen, keine `// ... rest bleibt gleich`
  3. Erkläre Electron-spezifische Entscheidungen kurz
  4. Teste gedanklich: Würde das mit `contextIsolation=true` funktionieren?
  5. Denke an Windows-Pfade: Backslashes in gespeicherten Pfaden beachten
- 



[HIER KONKRETE AUFGABE EINFÜGEN — z.B.]

"Baue den Hugging Face Modell-Browser ein:



- Neue Sidebar-Seite 'Modelle'
- Suche über HF API
- Download-Button mit Fortschrittsbalken
- Automatisches Erkennen von GGUF-Dateien"

---

Projektname: Hermes Control Center Version: 2.0.0 Stack: Electron 36+, Node.js, Vanilla JS, electron-store  
Zielformat: Windows 10/11 x64 Stand: [AKTUELLES DATUM]

[ ] text

---

## 📁 Aktualisiertes `package.json`

```json

```
{
  "name": "hermes-control-center",
  "version": "2.0.0",
  "description": "Desktop-Launcher und Kontrollzentrum für lokale KI-Modelle",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "dev": "electron . --inspect",
    "build": "electron-builder --win --x64",
    "pack": "electron-builder --dir",
    "release": "electron-builder --publish always"
  },
  "dependencies": {
    "electron-store": "^8.2.0",
    "electron-updater": "^6.1.7"
  },
  "devDependencies": {
    "electron": "^36.0.0",
    "electron-builder": "^24.13.0"
  },
  "build": {
    "appId": "com.hermes.controlcenter",
    "productName": "Hermes Control Center",
    "copyright": "Copyright © 2025",
    "win": {
      "target": [ { "target": "nsis", "arch": [ "x64" ] } ],
      "icon": "assets/icon.ico",
      "requestedExecutionLevel": "asInvoker"
    },
    "nsis": {
      "oneClick": false,
      "allowToChangeInstallationDirectory": true,
      "createDesktopShortcut": true,
```

```
    "createStartMenuShortcut": true,
    "shortcutName": "Hermes Control Center"
  },
  "publish": {
    "provider": "github",
    "owner": "DEIN_GITHUB_USERNAME",
    "repo": "hermes-control-center"
  },
  "files": [
    "main.js",
    "preload.js",
    "renderer/**",
    "assets/**",
    "node_modules/**",
    "!node_modules/.cache"
  ]
}
}
```



{ } Bash

```
npm install
npm start
```



| | | |
|----------------------------------|----------------------|---|
| Dateiauswahl (exe/gguf) | main.js + preload.js | ✓ |
| llama-server starten/stoppen | main.js | ✓ |
| Echtzeit-Logs (stdout/stderr) | main.js → IPC | ✓ |
| Log-Suche + Filter + Export | app.js | ✓ |
| Tray-Icon + Kontextmenü | main.js | ✓ |
| Autostart (setLoginItemSettings) | main.js | ✓ |
| Native Notifications | main.js | ✓ |
| Hermes Quick-Connect Panel | app.js | ✓ |
| API-Live-Test (Chat-Completion) | app.js | ✓ |

| | | |
|---------------------------------|------------|--------|
| Modell-Info (HF /v1/models) | app.js | ✓ |
| 5-Schritte Setup-Wizard | app.js | ✓ |
| Dashboard mit Status-Cards | app.js | ✓ |
| Settings-Page (Toggle-Switches) | app.js | ✓ |
| Persistenz (electron-store) | main.js | ✓ |
| Multi-Page SPA (Sidebar-Nav) | index.html | ✓ |
| Explorer-Integration | main.js | ✓ |
| Akzentfarben-Auswahl | app.js | ✓ |
| Uptime-Counter | app.js | ✓ |
| Auto-Updater | — | ⌚ Next |
| HF Modell-Browser | — | ⌚ Next |
| Chat-Interface | — | ⌚ Next |
| GPU-Monitor | — | ⌚ Next |
| Multi-Profil-System | — | ⌚ Next |

📌 Nächste Optionen:

- **A** — Auto-Updater (electron-updater + GitHub Releases)
- **B** — Hugging Face Modell-Browser direkt eingebaut
- **C** — Chat-Interface mit Streaming
- **D** — GPU/RAM-Monitor Panel